PEARLS AQI PREDICTOR

A Serverless MLOps Pipeline for 72-Hour Air Quality Forecasting

Project Report

Aleeza Rizwan

Data Science Intern
10 Pearls SHINE Cohort 2026

Live Dashboard: pearls-aqi-predictor.streamlit.app
GitHub Repository: github.com/its-aleezA/Pearls-AQI-Predictor

June 2026

Table of Contents

1. Introduction

Rawalpindi, Pakistan, ranks among South Asia's most polluted urban centres. Residents face AQI readings that routinely cross the **Unhealthy (151–200)** and **Very Unhealthy (201–300)** thresholds, particularly during winter months when temperature inversions trap particulate matter close to the ground. Despite the scale of the problem, publicly accessible, Rawalpindi-specific air quality forecasting tools are essentially nonexistent.

This project, Pearls AQI Predictor, addresses that gap. It is a fully automated, serverless MLOps system that ingests real-time weather and pollutant data, engineers a rich feature set, trains and compares multiple machine learning models, and serves 72-hour forward predictions through a styled, interactive web dashboard. Every part of the pipeline, from data ingestion to dashboard refresh, runs without any manual intervention.

The project was completed as part of a Data Science internship at 10 Pearls as part of their SHINE Cohort 2026.

1.1 Problem Statement

The core challenge this system addresses is: given the current and recent history of weather conditions and pollutant concentrations in Rawalpindi, can a machine learning model reliably predict the composite Air Quality Index 1 to 72 hours into the future? A working solution requires solving several sub-problems simultaneously:

- Continuous, automated data collection from external APIs without manual intervention
- Feature engineering that captures the temporal dynamics of pollution (lag effects, diurnal cycles, rolling trends)
- A reproducible, auditable training process with principled model selection
- A real-time dashboard accessible to any user without technical knowledge
- Infrastructure robust enough to run unattended in a cloud environment

1.2 Objectives

1. Build a complete MLOps pipeline, data ingestion, feature engineering, model training, inference, and deployment, as a cohesive, automated system
2. Demonstrate production-grade practices: Feature Store, Model Registry, CI/CD pipelines, secrets management, and graceful error handling
3. Generate forward-looking 72-hour AQI forecasts with hazard-level alerts calibrated to WHO thresholds
4. Provide model explainability through SHAP feature importance
5. Deploy a publicly accessible, auto-updating dashboard

2. System Architecture

The system is built around four loosely coupled pipelines, each with a distinct responsibility, connected through shared cloud infrastructure. The design is intentionally serverless, no persistent compute resources are required beyond what GitHub Actions provides on demand.

2.1 High-Level Overview

At the highest level, two GitHub Actions workflows govern the system. The **Feature Pipeline** runs every hour, fetching the latest AQI and weather data and pushing engineered features to a Hopsworks Feature Store. The **Training Pipeline** runs every day at 02:00 UTC (07:00 PKT), backfilling 60 days of historical data, retraining three candidate models, selecting the champion, generating a 72-hour forecast, and committing the output files back to the GitHub repository, which triggers an automatic redeploy of the Streamlit dashboard.

2.2 Component Breakdown

Component	Script(s)	Trigger	Output
Feature Pipeline	fetch_data.py, feature_engineering.py, upload_to_hopsworks.py	Hourly (GitHub Actions cron)	New row in Hopsworks Feature Store
Backfill Pipeline	backfill_data.py	Daily (start of Training Pipeline)	60 days of hourly features in Feature Store
Training Pipeline	train_aqi_model.py	Daily at 02:00 UTC	Champion model + SHAP plot in Model Registry
Inference Pipeline	batch_inference.py	Daily (after training)	Validation CSV + 72h forecast CSV + monitoring plot
Web Dashboard	app.py	Auto-deploy on git push	Live Streamlit app at public URL

2.3 Data Flow

The data flow follows a strict unidirectional path: external APIs feed the Feature Store, the Feature Store feeds the training and inference scripts, model artifacts go to the Model Registry, and inference outputs are committed to the repository which drives the dashboard.

One important design choice is the **fallback chain**: every script that reads from Hopsworks wraps the call in a try/except block. If the offline Feature Store has not finished materialising (a known behaviour of the Hopsworks free tier where Hudi file writes can lag by several minutes), the script falls back gracefully to a locally generated CSV. This ensures the pipeline never fails due to infrastructure timing issues.

3. Data Sources

Three external data sources feed the pipeline. All are free-tier or completely free. The system is designed so that any one source can fail gracefully without crashing the pipeline.

3.1 WAQI (World Air Quality Index)

The primary AQI data source. The WAQI API provides real-time readings from ground sensors. The feature pipeline queries the geo-coordinates endpoint (33.5973°N, 73.0479°E) to ensure it targets the Rawalpindi sensor rather than a neighbouring city's station.

Fields extracted: composite AQI, PM2.5 (μg/m³), PM10 (μg/m³), NO2 (ppb), O3 (ppb), SO2 (ppb), CO (ppm). Not all fields are always populated; sensors go offline, calibration windows create gaps. The feature engineering step handles these gracefully by treating missing pollutant readings as NaN and filling them with column medians during training.

3.2 OpenWeatherMap

Weather conditions are fetched from OpenWeatherMap's current weather endpoint. Weather variables are strong predictors of AQI: wind disperses pollutants, high humidity traps particulate matter, and temperature inversions (common in Rawalpindi winters) cause AQI spikes.

Fields extracted: temperature (°C), relative humidity (%), atmospheric pressure (hPa), wind speed (m/s).

3.3 Open-Meteo Historical Archive

For the backfill pipeline, historical weather data comes from the **Open-Meteo Archive API**, a completely free service with no API key requirement, covering years of hourly historical weather globally. This is a significant practical advantage over OpenWeatherMap's historical endpoint, which requires a paid plan.

The backfill fetches temperature, relative humidity, surface pressure, and wind speed at hourly granularity for the past 60 days. This is then merged with daily AQI readings from the WAQI backfill endpoint and expanded to hourly resolution.

Source	Variables	Granularity	History Depth	API Key
WAQI	AQI, PM2.5, PM10, NO2, O3, SO2, CO	Real-time (hourly append)	Current only (live fetch)	Yes (free)
OpenWeatherMap	Temp, Humidity, Pressure, Wind	Real-time (hourly append)	Current only (live fetch)	Yes (free)
Open-Meteo Archive	Temp, Humidity, Pressure, Wind	Hourly	Years	No

Source	Variables	Granularity	History Depth	API Key
WAQI Backfill	AQI, PM2.5 (daily avg)	Daily	~90 days	Yes (free)

4. Feature Engineering

Feature engineering is the highest-leverage step in any time-series forecasting pipeline. The quality of the feature set determines the ceiling on model performance far more than the choice of algorithm. This pipeline engineers 23 features from the 7 raw API variables, grouped into four categories.

4.1 Time-Based Features

Pollution in urban environments follows strong daily and weekly cycles. Industrial activity, traffic, and burning patterns all correlate with the time of day and day of week. These features allow the model to learn those cyclic patterns without access to a calendar.

Feature	Description	Rationale
hour	Hour of day (0–23)	Captures diurnal cycle; rush hour vs. midnight
day_of_week	Day (0=Mon, 6=Sun)	Weekday vs. weekend pollution patterns differ significantly
month	Month of year (1–12)	Seasonal variation; winter inversions, monsoon washout
is_weekend	Binary (0/1)	Compressed encoding of weekday/weekend distinction

4.2 Lag Features

Lag features are the most important additions to the raw data. Air quality has strong **temporal autocorrelation**: if AQI has been 180 for the past three hours, the next hour is far more likely to be close to 180 than to suddenly drop to 50. Without lag features, the model has no memory; it must predict from static meteorological conditions alone, which is far weaker.

Feature	Description	Why this lag specifically
aqi_lag_1	AQI at t-1 hour	Immediate momentum; strongest single predictor
aqi_lag_2	AQI at t-2 hours	Short-term trend detection
aqi_lag_3	AQI at t-3 hours	Extends trend window to cover typical rush hour duration
aqi_lag_24	AQI at t-24 hours	Same-time-yesterday; captures diurnal repeatability

4.3 Rolling Statistics

Rolling statistics summarise the recent trajectory of AQI rather than just its last known value. The rolling mean captures sustained trends; the rolling standard deviation captures volatility. A sudden spike in standard deviation often signals an anomalous pollution event.

Feature	Window	Captures
aqi_rolling_6h_mean	6 hours	Short-term sustained trend
aqi_rolling_24h_mean	24 hours	Daily baseline; filters out short spikes
aqi_rolling_6h_std	6 hours	Short-term volatility / event detection
aqi_rolling_24h_std	24 hours	Day-level stability of air quality

4.4 Change Rate

aqi_diff: the first-order difference of AQI (current minus previous). This encodes the *direction and magnitude of change*; whether AQI is rising or falling and how fast. It is a simple but powerful derivative feature that complements the lag and rolling features.

4.5 Implementation Notes

The `feature_engineering.py` function sorts the dataframe chronologically before computing any features, which is non-negotiable; computing a lag on unsorted data produces nonsensical values. All NaN values introduced by lags and rolling windows at the start of the series are handled downstream: the training script drops rows where core features (AQI, temperature, humidity) are null, then fills remaining NaNs with column medians.

5. Machine Learning Models

Three classical ML models and one deep learning model are trained and compared on every daily run. The system is designed so that the champion changes automatically as data accumulates and model performance evolves.

5.1 Model Evaluation Strategy

The evaluation uses **TimeSeriesSplit(n_splits=5)** from scikit-learn rather than the more common random train-test split. This distinction matters enormously for time-series data.

With a standard random split, rows from the future end up in the training set. The model effectively learns to use future information to predict the past, a form of data leakage that produces inflated metrics on the training/validation set but catastrophically poor performance in deployment. TimeSeriesSplit avoids this by always training on the chronologically earlier data and evaluating on the later data across five rolling folds.

Each fold uses a StandardScaler fit only on the training split and applied to the test split; fitting the scaler on the entire dataset before splitting would constitute another, subtler form of leakage.

Three metrics are reported per model: RMSE (root mean squared error), MAE (mean absolute error), and R^2. The champion is selected on RMSE. All metrics are stored in the Hopsworks Model Registry alongside the model artifact.

5.2 Random Forest Regressor

Configuration: n_estimators=200, random_state=42, n_jobs=-1

An ensemble of 200 decision trees, each trained on a random subsample of features and rows. Random Forest is robust to outliers, handles non-linear interactions between lag features and weather variables naturally, and provides native support for SHAP TreeExplainer, an exact, computationally efficient method for computing feature attributions.

With sufficient data, Random Forest typically outperforms Ridge on this type of time-series problem because the relationship between rolling statistics and AQI is non-linear. At current data volumes (approximately 70 usable rows after cleaning), the higher variance of tree-based methods puts them at a disadvantage.

5.3 Gradient Boosting Regressor

Configuration: n_estimators=200, random_state=42

A sequential ensemble where each tree corrects the residuals of the previous one. Gradient Boosting generally achieves lower bias than Random Forest at the cost of higher variance, and is more sensitive to hyperparameter choices. Like Random Forest, it supports TreeExplainer for exact SHAP computation. It tends to be the strongest performer on medium-sized, well-featured tabular datasets, likely to become the champion once the hourly pipeline has accumulated several weeks of data.

5.4 Ridge Regression

Configuration: alpha=1.0

L2-regularised linear regression. Ridge acts as a strong, theoretically grounded baseline. Its inductive bias, assuming a linear relationship between features and target, is often a reasonable approximation for AQI forecasting over short horizons, particularly when data is limited. Ridge is the current champion model precisely because of this: with only ~70 training samples, the simplicity of linear regression generalises better than tree-based methods that require more data to reliably discover non-linear structure.

When Ridge is champion, permutation importance is used as a fallback for feature importance since TreeExplainer is not applicable to linear models.

5.5 LSTM Deep Learning Model

Architecture: LSTM(64) → Dropout(0.2) → LSTM(32) → Dropout(0.2) → Dense(1)

A two-layer LSTM network trained on sliding windows of 24 hours; each input sequence covers the past 24 hourly feature vectors, and the target is the AQI at the next hour. This architecture is appropriate for time-series data because LSTMs maintain a hidden state across the sequence, allowing them to capture long-range dependencies that fixed-lag features might miss.

The LSTM is trained with early stopping (patience=5) to prevent overfitting on small datasets. It is skipped gracefully if TensorFlow is not installed in the environment; the pipeline continues with the three classical models. At current data volumes the LSTM is unlikely to be competitive, but it will be evaluated automatically as data grows.

6. Model Performance

Performance metrics reflect the current stage of the project: a recently deployed pipeline with approximately 70 usable training rows after cleaning. These numbers will improve substantially as the hourly pipeline accumulates data over coming weeks.

6.1 Current Metrics

Model	Avg RMSE	Avg MAE	Avg R^2	Status
Random Forest	20.12	14.02	-0.141	Evaluated
Gradient Boosting	21.97	15.73	-0.145	Evaluated
Ridge Regression	12.90	7.60	-0.133	Champion
LSTM	N/A	N/A	N/A	Skipped (TF not installed)

6.2 Interpreting Negative R^2

A negative R^2 indicates the model performs worse than a naive predictor that always outputs the mean of the target variable. This is the expected result at low data volumes and is **not a bug or a failure**.

With TimeSeriesSplit(5) and 70 samples, each training fold contains roughly 12–14 rows. No model, linear or non-linear, can learn meaningful patterns from 12 training samples, particularly with 22 input features. The negative R^2 reflects this data scarcity rather than any flaw in the modelling approach.

NOTE Ridge Regression's lower RMSE (12.90 vs 20+ for the tree models) is consistent with the bias-variance tradeoff: linear models generalise better under severe data constraints because they have fewer parameters to fit. As data accumulates, this advantage will erode and the tree-based models are expected to surpass Ridge.

6.3 Expected Trajectory

Based on standard learning curve behaviour for these model classes on AQI forecasting tasks, the following trajectory is expected as the hourly pipeline accumulates data:

Data Volume	Expected Champion	Expected RMSE Range	Expected R^2
Current (~70 rows)	Ridge Regression	12–20	Negative
1 week (~168 rows)	Ridge or Gradient Boosting	8–15	0.0 to 0.3
1 month (~720 rows)	Gradient Boosting or LSTM	5–10	0.4 to 0.65
3+ months (2000+ rows)	LSTM or Gradient Boosting	<8	> 0.65

The daily retraining pipeline will detect these shifts automatically ; no code changes are required for the champion to change.

7. Explainability (SHAP Analysis)

Model explainability is not an afterthought in this system; it is a first-class deliverable, surfaced directly in the dashboard's Explainability tab. The use of SHAP reflects a genuine commitment to understanding why the model makes the predictions it does, not just evaluating whether its numbers are good.

7.1 SHAP Method Selection

For tree-based champions (Random Forest, Gradient Boosting), **shap.TreeExplainer** is used. This is an exact method; it does not sample or approximate. It computes the true Shapley value for each feature in each prediction, exploiting the tree structure for computational efficiency. The result is a theoretically grounded attribution: each feature's SHAP value represents the exact contribution of that feature to the difference between the model's prediction and its baseline (mean) prediction.

For Ridge Regression (the current champion), permutation importance is used as a fallback. Permutation importance is model-agnostic: it measures the drop in model performance when a feature's values are randomly shuffled, which breaks the feature's relationship with the target.

7.2 Expected Feature Rankings

Based on domain knowledge of AQI dynamics and the feature set design, the following feature importance ranking is expected once sufficient data is available:

Rank	Feature	Direction	Reasoning
1	aqi_lag_1	Positive	Yesterday's AQI is the strongest predictor of today's; air quality is highly persistent
2	aqi_rolling_24h_mean	Positive	Sustained pollution episodes last days; the 24h mean captures ongoing events
3	aqi_lag_24	Positive	Same-time-yesterday captures the diurnal cycle of traffic and industrial activity
4	wind_speed	Negative	Higher wind disperses pollutants; a critical mechanical driver of AQI reduction
5	humidity	Positive	High humidity traps fine particulate matter close to ground level
6	hour	Mixed	Rush hour peaks (08:00, 18:00) vs. clean early morning hours
7	aqi_rolling_6h_mean	Positive	Short-term trend; captures pollution events building over hours
8	aqi_diff	Mixed	Rising or falling AQI trajectory; momentum signal

7.3 Reading the SHAP Summary Plot

The SHAP summary plot in the dashboard shows one dot per prediction per feature. Dots are coloured by the feature's value (red = high, blue = low) and positioned on the x-axis by their SHAP value (positive = pushes prediction higher, negative = pushes lower). Features are ranked vertically by mean absolute SHAP value.

A red dot with a large positive SHAP value means: this prediction had a high value for this feature, and that pushed the predicted AQI up. A blue dot with a large negative SHAP value means: this prediction had a low value for this feature, and that pushed the predicted AQI down. The spread of dots along the x-axis indicates the consistency of the effect across predictions.

8. CI/CD Automation

The automation layer is what distinguishes this project from a notebook-based ML experiment. Two GitHub Actions workflows replace what would otherwise require a persistently running server or daily manual intervention.

8.1 Feature Pipeline (Hourly)

File: `.github/workflows/feature_pipeline.yml` | **Cron:** `0 * * * *`

The hourly workflow runs on GitHub's free-tier Linux runners. Each run spins up a fresh Ubuntu container, installs dependencies, and executes the three-script fetch-engineer-upload sequence. The runner is destroyed after each run; there is no persistent state, no server to maintain, and no cost.

Steps in order:

1. Check out the repository
2. Set up Python 3.11 with pip cache for faster dependency installation
3. Install requirements.txt, then pin hopsworks==4.7.* and confluent-kafka (required for Kafka-based Feature Store writes)
4. Run fetch_data.py, it queries WAQI and OWM, appends one row to aqi_history.csv
5. Run feature_engineering.py, it reads aqi_history.csv, computes all 23 features, writes aqi_features.csv
6. Run upload_to_hopsworks.py, it casts dtypes, upserts to Feature Store version 2

8.2 Training Pipeline (Daily)

File: `.github/workflows/training_pipeline.yml` | **Cron:** `0 2 * * *`

The daily workflow is more complex, covering the full model lifecycle from data backfill to dashboard update:

1. Check out repository
2. Set up Python 3.11
3. Install dependencies
4. Run backfill_data.py, it fetches 60 days of historical AQI (WAQI) and weather (Open-Meteo), merges, engineers features, pushes to Feature Store
5. Run feature_engineering.py on the fresh fetch
6. Run train_aqi_model.py, it reads features, runs TimeSeriesSplit evaluation, selects champion, computes SHAP, uploads model to registry
7. Run batch_inference.py, it downloads model, generates historical validation predictions and 72-hour autoregressive forecast, saves CSVs and monitoring plot
8. git add predictions/*.csv models/*.png, it stages the output artifacts
9. git commit -m 'Automated sync [skip ci]', it commits the outputs; [skip ci] prevents the commit from triggering another workflow run
10. git push origin main, it triggers Streamlit Community Cloud to redeploy the dashboard with fresh data

8.3 Secrets Management

All API keys are stored as GitHub Repository Secrets (Settings > Secrets and variables > Actions > Repository secrets). They are injected as environment variables during workflow runs and are never written to logs, committed to the repository, or exposed in any output. The local .env file is excluded from version control via .gitignore. The .env.example file documents which variables are required without containing any real values.

KEY DESIGN

[skip ci] on the automated commit is non-negotiable. Without it, the git push at the end of the training pipeline would trigger the training pipeline again, creating an infinite loop of daily runs. The tag instructs GitHub Actions to ignore the commit when evaluating workflow triggers.

9. The 72-Hour Autoregressive Forecast

Generating a forecast 72 hours into the future from a model trained to predict one hour ahead requires an autoregressive approach: the model's own predictions are fed back as inputs for subsequent steps. This introduces compounding uncertainty but is the only tractable approach when future weather inputs are not available.

9.1 The Window History Approach

A naive implementation would simply take the last known feature row, update the time-based features, and call the model 72 times. This fails because the lag features and rolling statistics would remain frozen at their last observed values; aqi_lag_1 would be the same for all 72 predictions, and aqi_rolling_6h_mean would never update to reflect the predictions being made.

The correct approach uses a **sliding window_history buffer**, seeded with the last 24 actual AQI values from the Feature Store. At each forecast step:

1. Pull the trailing 6 and 24 values from window_history to compute rolling mean and std
2. Pull the last 1, 2, 3 values for lag features (aqi_lag_1, aqi_lag_2, aqi_lag_3)
3. Pull window_history[-24] for aqi_lag_24 (or the earliest available value if less than 24 steps have been made)
4. Update time-based features (hour, day_of_week, month, is_weekend) for the future timestamp
5. Carry forward the last known weather values (temperature, humidity, pressure, wind speed)
6. Assemble the complete 23-feature input vector, scale it, and call model.predict()
7. Append the prediction to window_history so subsequent steps use it as context

This approach ensures that the feature vectors presented during inference are mathematically consistent with those used during training; the rolling statistics and lag features update at every step rather than being frozen at their last observed values.

9.2 Limitations of the Autoregressive Approach

The main weakness of this approach is error propagation. If the model overestimates AQI at hour 5, that overestimate becomes aqi_lag_1 at hour 6, inflating the hour 6 prediction, which then inflates hour 7, and so on. Over 72 steps, small systematic biases can compound into large forecast errors.

The dashboard forecast charts showing high AQI values (above 300) in the 48–72 hour range reflect this: the model is currently trained on data where AQI was often high, and the autoregressive propagation of those high lag values keeps pushing predictions upward. This will self-correct as the model accumulates more diverse training data spanning different pollution conditions.

Integrating a weather forecast API (such as Open-Meteo's free forecast endpoint) would partially address this by providing genuine future temperature and wind speed values rather than carrying forward the last known values. This is identified as a priority for future work.

10. Web Dashboard

The dashboard is built with Streamlit and deployed on Streamlit Community Cloud at **pearls-aqi-predictor.streamlit.app**. It uses a custom colour palette and typography to move distinctly away from the default Streamlit aesthetic.

10.1 Visual Design

The colour palette (Light Blue #B8D8D8, Cool Steel #7A9E9F, Blue Slate #4F6367, Beige #EEF5DB, Coral #FE5F55) was chosen to convey both environmental context (blues and greens evoking air and water) and urgency (coral for alerts and critical metrics). The beige background is easier on the eyes than pure white for a monitoring dashboard that users may leave open for extended periods.

Typography uses Playfair Display (a classical serif) for headings and large numbers, and DM Sans (a geometric sans-serif) for body text. This editorial combination avoids the generic feel of most data dashboards.

A city skyline SVG in the header dynamically shifts its sky gradient colour based on the forecasted AQI; teal-blue for clean air, warm ochre for moderate, orange-red for unhealthy, and dark burgundy for hazardous. This gives an at-a-glance visual cue before any numbers are read.

10.2 Tab Structure

Tab	Contents	Key Feature
72-Hour Forecast	Hazard alert banner, 4 metric cards, timeline chart, day-by-day summary	Alert banner colour matches WHO AQI category
Model Validation	Latest actual vs. predicted metrics, historical line chart, monitoring plot	Error metric colour-coded by magnitude
Explainability	SHAP summary plot with interpretive text	Reads from models/shap_summary.png committed by training pipeline
Raw Data	Full prediction log and forecast table, CSV download buttons	Users can export data without any code

10.3 Hazard Alert System

The dashboard implements a four-level hazard alert system calibrated to WHO and US EPA AQI thresholds:

AQI Range	Category	Alert Style	Advisory
0–100	Good / Moderate	Green success banner	Air quality is forecast to remain acceptable

AQI Range	Category	Alert Style	Advisory
101–150	Unhealthy for Sensitive Groups	Yellow warning	Limit outdoor time for children, elderly, and those with respiratory conditions
151–200	Unhealthy	Orange warning	General public should reduce prolonged outdoor exertion
201–300	Very Unhealthy	Red error banner	Avoid outdoor activity; sensitive groups should stay indoors
300+	Hazardous	Dark red error	Avoid all outdoor activity; wear N95 masks even indoors if possible

10.4 Auto-Update Mechanism

The dashboard does not poll the Feature Store or call any API at runtime. Instead, it reads from two CSV files committed to the repository (predictions/aqi_batch_predictions.csv and predictions/aqi_72h_forecast.csv). When the daily training pipeline runs and pushes updated CSVs, Streamlit Community Cloud detects the git push and automatically redeploys the app, typically within 2–3 minutes. The net result is a dashboard that updates daily without any server-side logic at the app level.

11. Hopsworks Integration

Hopsworks provides both the Feature Store and the Model Registry that anchor the pipeline's cloud infrastructure. Using these components elevates the project from a script-based ML experiment to a production-grade MLOps system with auditability, versioning, and reproducibility.

11.1 Feature Store

Features are stored in a Hopsworks Feature Group named `aqi_predictions`, version 2 (version 1 had an incompatible schema from earlier development). The feature group uses `timestamp` as the primary key; each row represents one hour of data for Rawalpindi.

The version 2 schema reflects the full 23-feature set after all feature engineering improvements. The Hopsworks Python SDK handles schema validation at insert time, which is why dtype casting (all nullable columns to float64, integer columns to int64) is performed explicitly in upload_to_hopsworks.py and backfill_data.py before any insert call.

Data is written to the Feature Store via Kafka (the hopsworks[python] extra installs the confluent-kafka dependency required for this). The offline store materialises the data into Hudi files on HDFS; this process can lag by 2–5 minutes on the free tier, which is why all reading scripts implement the fallback chain described in Section 2.3.

11.2 Model Registry

After each training run, the champion model, scaler, and SHAP plot are uploaded to the Hopsworks Model Registry under the name aqi_prediction_model. Each training run creates a new version, and the batch inference script always fetches the highest available version dynamically (using mr.get_models() to find the max version number) rather than hardcoding a version. This ensures the inference pipeline automatically uses the latest champion without any manual updates.

Metrics stored per model version: RMSE, MAE, R^2 (sanitised to replace NaN/Inf with 0.0 before upload, since Hopsworks rejects non-numeric JSON values in the metrics dict). A full description string includes the champion model name, all candidate model results, and the feature list.

12. Key Design Decisions

Several choices in this project are non-obvious and worth explaining explicitly. Each represents a deliberate trade-off rather than a default.

12.1 TimeSeriesSplit Over Random Split

This is the most consequential modelling decision in the pipeline. Using sklearn's default train_test_split with shuffle=True would allow future AQI values to appear in the training set, inflating evaluation metrics and producing a model that underperforms in deployment. TimeSeriesSplit strictly prevents this by training on the chronologically earlier portion and testing on the later portion at each fold.

12.2 Fallback Chain for Infrastructure Resilience

Every script that reads from Hopsworks is wrapped in a try/except block. Rather than failing loudly when the offline store has not finished materialising, the pipeline falls back to locally generated CSV files. This design choice means the daily training run completes successfully even when Hopsworks is slightly delayed, an important property for an automated system that runs unattended.

12.3 Window History Buffer for Autoregressive Inference

The 72-hour forecast loop maintains a sliding list of recent AQI values (both actual and predicted) and recomputes rolling statistics and lag features at every step. The alternative, freezing the last known feature vector and only updating time-based fields, would produce increasingly inaccurate predictions as the forecast horizon extends, because the lag features would never reflect the model's own previous outputs.

12.4 No Model Binaries in Version Control

Trained model files (.pkl, .keras) are excluded from the git repository via .gitignore. They live exclusively in the Hopsworks Model Registry, which provides versioning, metric tracking, and artifact management far superior to what git provides for binary files. This keeps the repository lightweight and avoids binary diff noise in git history.

12.5 Dtype Casting Before Hopsworks Insert

Hopsworks enforces strict schema compatibility: inserting a DataFrame where a column has dtype 'null' (all Python None values) or 'int' instead of 'double' raises a FeatureStoreException. All upload and backfill scripts explicitly cast columns to their correct types, float64 for continuous variables and int64 for categorical/binary ones, immediately before any insert call. This is not elegant, but it is the correct way to handle the mismatch between Python's dynamic type system and Hopsworks' typed schema.

13. Limitations and Future Work

13.1 Current Limitations

Data volume: The most significant current constraint. WAQI's free tier provides real-time data and a limited backfill, resulting in approximately 70 usable training rows after feature engineering cleaning. Model performance metrics will not be meaningful until at least 2–3 weeks of hourly data have accumulated.

Historical AQI resolution: The WAQI backfill provides daily average AQI values, which are expanded to 24 identical hourly rows per day. This expansion introduces artificial repetition in the training data and does not reflect genuine intra-day variation. A proper solution would require access to hourly historical AQI measurements, which require either a paid API or a local sensor network.

Future weather inputs: The 72-hour forecast carries forward the last known weather values (temperature, wind speed, etc.) rather than using forecast values. Integrating the Open-Meteo Forecast API (free, no key required) would provide genuine 7-day weather forecasts as inputs, materially improving forecast accuracy at longer horizons.

Single city: The pipeline is hardcoded to Rawalpindi (33.5973°N, 73.0479°E). The code is parameterised enough that extending to additional cities would require minimal changes, but the current deployment only serves one location.

13.2 Priority Future Work

1. Weather forecast integration: feed Open-Meteo's free 7-day forecast into the inference window rather than carrying forward last known values
2. Multi-city support: extend to Lahore, Karachi, and Islamabad with a city selector in the dashboard
3. AQI alert notifications: email or SMS alerts when hazardous AQI is forecast (Twilio / SendGrid)
4. Model drift monitoring: track how RMSE evolves over time and trigger automatic retraining if performance degrades beyond a threshold
5. AQI subtype decomposition: separate PM2.5, NO2, and O3 sub-indices rather than predicting composite AQI only, enabling more targeted public health advisories
6. Sensor-level data integration: connect to low-cost IoT AQI sensors (PurpleAir, Shinyei) for hourly intra-city variation at higher resolution

14. Conclusions

The Pearls AQI Predictor demonstrates that a production-grade MLOps system (one with automated data ingestion, a cloud feature store, a model registry, CI/CD pipelines, and a live dashboard) is achievable within the constraints of free-tier infrastructure and a student internship timeline.

Several technical contributions are worth noting specifically. The TimeSeriesSplit-based evaluation framework prevents data leakage that would otherwise inflate reported metrics. The window_history sliding buffer produces a mathematically consistent 72-hour autoregressive forecast. The fallback chain makes the system resilient to cloud infrastructure delays without requiring manual intervention. The SHAP integration produces genuinely interpretable feature attributions rather than black-box predictions.

The current model metrics are limited by data volume, a problem the hourly accumulation pipeline is actively solving. The infrastructure is designed so that as data grows, model performance improves automatically without any code changes. This is the core value proposition of MLOps over traditional ML: the system learns from its environment continuously.

Air quality forecasting for Rawalpindi has direct public health relevance. On days when AQI climbs above 200, outdoor exposure causes measurable respiratory harm, particularly for children and the elderly. A reliable 72-hour forecast gives residents, schools, and public health authorities advance warning to take protective action. That the system generating these forecasts runs entirely on free-tier infrastructure means it could be replicated and extended to any city in Pakistan, or indeed anywhere in the world, without significant cost.

15. Technology Stack and References

15.1 Full Technology Stack

Layer	Tool / Service	Version / Tier	Purpose
Data ingestion	WAQI API	Free tier	Real-time AQI + pollutant data
Data ingestion	OpenWeatherMap API	Free tier	Real-time weather data
Data ingestion	Open-Meteo Archive	Free (no key)	Historical weather backfill
Feature Store	Hopsworks	Free tier	Feature versioning and retrieval
Model Registry	Hopsworks	Free tier	Model artifact and metrics storage
CI/CD	GitHub Actions	Free tier	Hourly and daily pipeline automation
ML	scikit-learn 1.4+	Open source	Random Forest, Gradient Boosting, Ridge, TimeSeriesSplit
Deep learning	TensorFlow / Keras	Open source	LSTM model
Explainability	SHAP 0.45+	Open source	TreeExplainer and permutation importance
Data processing	pandas 2.x, numpy 1.26+	Open source	Feature engineering and data manipulation
Visualisation	matplotlib, seaborn	Open source	EDA and monitoring plots
Dashboard	Streamlit 1.35+	Open source	Interactive web application
Hosting	Streamlit Community Cloud	Free tier	Public dashboard deployment
Secrets	GitHub Secrets + python-dotenv	Built-in	API key management
Serialisation	joblib	Open source	Model and scaler persistence
Kafka client	confluent-kafka	Open source	Feature Store write transport

15.2 Key References

- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. NeurIPS.
- Bergmeir, C., & Benitez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. Information Sciences, 191, 192-213.
- WAQI Technical Alliance. World Air Quality Index API. https://aqicn.org/api/
- Hopsworks AB. Hopsworks Feature Store Documentation. https://docs.hopsworks.ai/

- Open-Meteo. Historical Weather API. https://open-meteo.com/en/docs/historical-weather-api
- US EPA. Air Quality Index — A Guide to Air Quality and Your Health (2014).

Appendix A: Repository Structure

The complete file tree of the deployed repository:

File / Directory	Description
.github/workflows/feature_pipeline.yml	Hourly GitHub Actions workflow; fetch, engineer, upload
.github/workflows/training_pipeline.yml	Daily GitHub Actions workflow; backfill, train, infer, commit
.streamlit/config.toml	Streamlit theme configuration; palette, font settings
predictions/aqi_batch_predictions.csv	Historical actual vs. predicted AQI (auto-committed daily)
predictions/aqi_72h_forecast.csv	72-hour forward forecast (auto-committed daily)
predictions/aqi_inference_plot.png	Monitoring plot (auto-committed daily)
models/shap_summary.png	SHAP feature importance plot (auto-committed daily)
app.py	Streamlit dashboard; 4 tabs, custom styling, SVG skyline
fetch_data.py	API fetcher; WAQI and OpenWeatherMap, error handling
feature_engineering.py	Feature computation; 23 features from 7 raw variables
upload_to_hopsworks.py	Feature Store upserter; dtype casting and upsert logic
backfill_data.py	Historical backfill; 60 days of AQI and weather
train_aqi_model.py	Training pipeline; TimeSeriesSplit, 3+ models, SHAP, registry
batch_inference.py	Inference pipeline; window_history buffer, 72h forecast
download_model.py	Utility; download latest model from registry to local
test_api.py	Utility; verify all 4 API connections are functional
AQI_Analysis.py	EDA; 8 analysis plots: trends, correlation, temporal patterns
requirements.txt	Python dependencies for Streamlit Cloud deployment
.env.example	Template documenting required environment variables
.gitignore	Excludes .env, venvs, .pkl files, generated CSVs

Appendix B: GitHub Actions Cron Reference

Workflow	Cron Expression	UTC Time	PKT Time
Feature Pipeline	0 * * * *	Every hour on the hour	Every hour on the hour
Training Pipeline	0 2 * * *	02:00 daily	07:00 daily

Appendix C: AQI Category Thresholds

AQI Range	Category	Health Implications	Dashboard Alert
0–50	Good	Air quality satisfactory; negligible risk	Green success
51–100	Moderate	Acceptable; unusually sensitive people may be affected	Green success
101–150	Unhealthy for Sensitive Groups	Children, elderly, and people with respiratory disease at risk	Yellow warning
151–200	Unhealthy	General public may begin to experience health effects	Orange warning
201–300	Very Unhealthy	Health alert; everyone may experience serious effects	Red error
301–500	Hazardous	Emergency conditions; entire population likely affected	Dark red error